

CAPSTONE PROJECT REPORT

Selenium Python Automation Framework

Automated Login and Product Search using PyTest, Unittest and Page Object Model

Application Under Test

Automation Exercise

Submitted by

Pratyay Chowdhury

Enrl no-12023052020034

B.Tech – Computer Science & Engineering (Artificial Intelligence)
Institute of Engineering and Management, Kolkata

September 2026

Table of Contents

Table of Contents	2
1. Introduction	4
1.1 Project Context.....	4
1.2 Scope	4
2. Problem Statement	5
2.1 Objective	5
2.2 Expected Outcome	5
3. Application Under Test	6
3.1 Homepage	6
3.2 Login Interface.....	6
4. Tools and Technologies	7
4.1 Dependency Configuration.....	7
5. Framework Architecture	8
5.1 Architectural Layers.....	8
5.2 Project Structure	8
6. Configuration Management	9
7. Driver Factory and Browser Setup	10
7.1 Browser Lifecycle	10
7.2 Robustness	10
8. Page Object Model	11
8.1 LoginPage	11
8.2 ProductsPage	11
9. Base Page and Wait Utility	12
10. Login Automation	13
10.1 Successful Login Evidence	13
11. Product Search Automation	14
11.1 Products Page	14
11.2 Search Result Evidence.....	14
12. CSV Test Data Handling	15
13. PyTest and Unittest Integration.....	16
13.1 Unittest Component	16
14. Logging	17
15. Screenshot on Failure	18
16. HTML Reporting.....	19

17. Test Cases	20
17.1 Test Flow	20
18. Execution Results	21
18.1 Result Summary	21
19. Challenges and Solutions	22
20. Security and Good Practices	23
20.1 Repository Organization	23
21. Conclusion	24
21.1 Result	24
22. Future Scope	25
23. References	26
Appendix A – Key Framework Configuration	27
Appendix B – Submission Checklist	27

1. Introduction

Web application automation reduces repetitive manual verification by executing repeatable browser interactions through a programmable test framework. Selenium WebDriver provides browser control, while a test framework such as PyTest provides test discovery, fixtures, parameterization and reporting. A maintainable automation solution separates test intent from page interaction details so that changes to the application can be localized.

This project implements a Selenium Python automation framework for the Login and Product Search functionality of the Automation Exercise e-commerce practice website. The framework was developed around the requirements of the assigned Selenium Python Framework Development capstone, which specifies Unittest, PyTest, Page Object Model (POM), utility classes, configuration management, CSV test data, screenshots on failure and HTML reporting.

[filecite turn3file0](#)

The implementation focuses on a compact but reusable architecture rather than a single monolithic test script. Browser creation, waiting, configuration, logging, test-data loading and page interactions are placed in dedicated components.

1.1 Project Context

- Project type: Individual Selenium Python automation framework.
- Application: Automation Exercise.
- Primary automated flow: Launch browser → Login → Open Products → Search product → Verify search result.
- Primary framework technologies: Selenium WebDriver, PyTest and Unittest.
- Data source: CSV test data.
- Evidence: execution logs, screenshots and self-contained HTML test report.

1.2 Scope

The scope is intentionally limited to the Login and Product Search scenario specified for Assignment 2. Shopping-cart operations, quantity updates and cart verification are not part of this implementation because those belong to the separate capstone scenario in the assignment document. [filecite turn3file0](#)

2. Problem Statement

A basic Selenium script can automate a browser flow, but a collection of standalone scripts becomes difficult to maintain when locators, waits, browser settings, test data or application URLs change. The project therefore addresses the need for a structured framework that can execute the assigned login and product-search scenario while keeping reusable automation concerns separate from individual test cases.

2.1 Objective

- Develop a reusable Selenium Python automation framework.
- Apply Page Object Model to separate page interactions from test logic.
- Use pytest for test execution, fixtures and parameterization.
- Include unittest-based validation within the project.
- Centralize browser and timeout configuration.
- Read test data from CSV rather than hard-coding the search data in the test.
- Capture screenshots automatically when a test fails.
- Generate an HTML execution report.
- Provide logging for significant framework and test events.

2.2 Expected Outcome

The expected outcome is a framework in which the complete test suite can be executed from the project root with a single pytest command, while producing readable console results and an HTML report. The latest recorded execution completed all four collected tests successfully.

3. Application Under Test

Automation Exercise is a public e-commerce practice website designed for automation testing. The implemented scenario uses its login page and product catalogue/search functionality.

3.1 Homepage

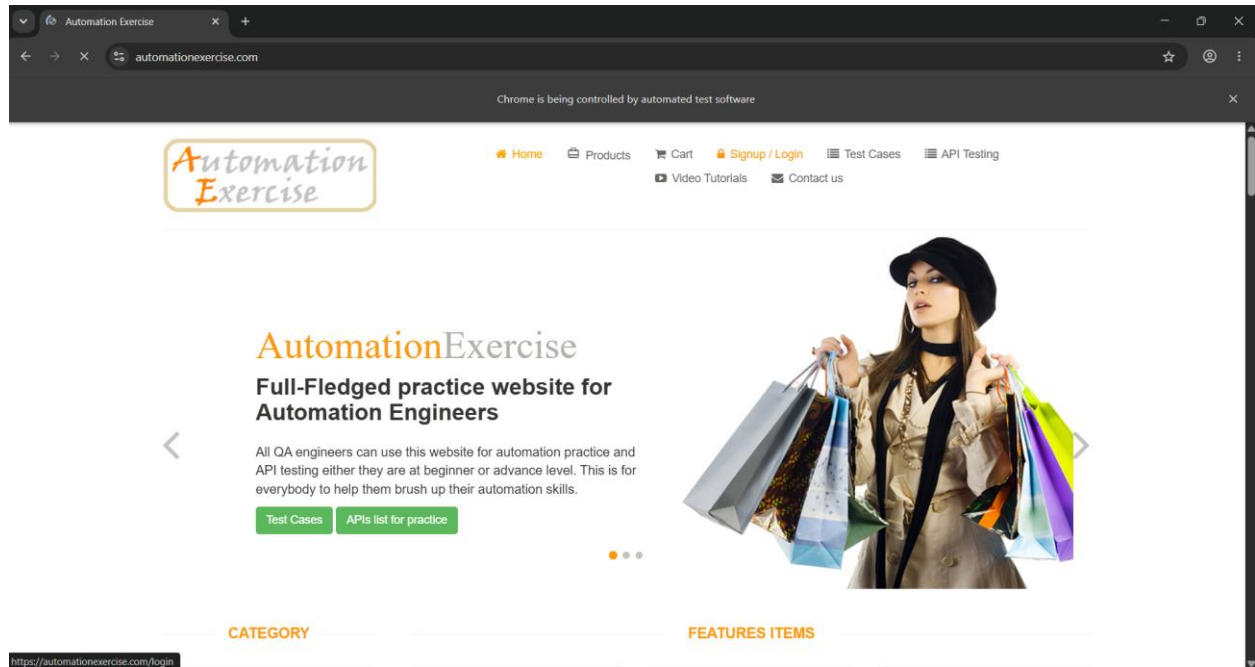


Figure 1. Automation Exercise homepage used as the application entry point.

3.2 Login Interface

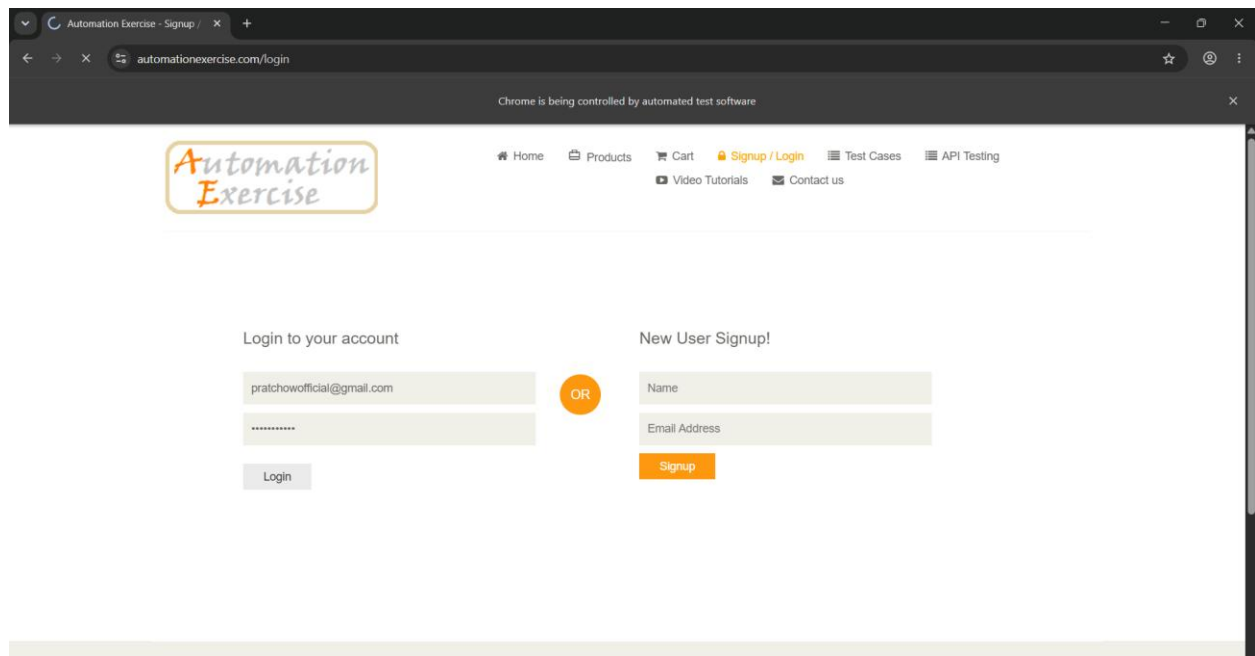


Figure 2. Login interface of the application under test.

4. Tools and Technologies

Technology / Tool	Purpose	Use in Project
Python 3.11.4	Programming language	Framework implementation and tests
Selenium 4.44.0	Browser automation	WebDriver control and element interaction
PyTest 9.1.1	Test framework	Discovery, fixtures, parameterization and execution
Unittest	Python test framework	Configuration validation component
pytest-html 4.2.0	Reporting plugin	Self-contained HTML execution report
webdriver-manager 4.0.2	Driver management	ChromeDriver installation/management
CSV	Test data format	Parameterized email/search-product data
VS Code	Development environment	Project implementation and execution
Git / GitHub	Version control and submission	Individual project repository

4.1 Dependency Configuration

```
pytest==9.1.1
pytest-html==4.2.0
pytest-metadata==3.1.1
selenium==4.44.0
webdriver-manager==4.0.2
python-dotenv==1.1.1
```

5. Framework Architecture

The framework follows a layered structure. Tests contain business-level verification, Page Objects encapsulate application interactions, utilities provide reusable services, configuration stores environment values, and the fixture manages browser lifecycle and failure evidence.

5.1 Architectural Layers

- Test Layer – test_browser.py, test_login_search.py and test_unittest_component.py.
- Page Layer – LoginPage, ProductsPage and BasePage.
- Utility Layer – driver factory, configuration reader, CSV reader, waits and logger.
- Configuration Layer – config.ini for URL, browser settings and timeout values.
- Data Layer – test_data.csv for parameterized test inputs.
- Execution/Evidence Layer – PyTest fixture, logs, screenshots and HTML report.

5.2 Project Structure

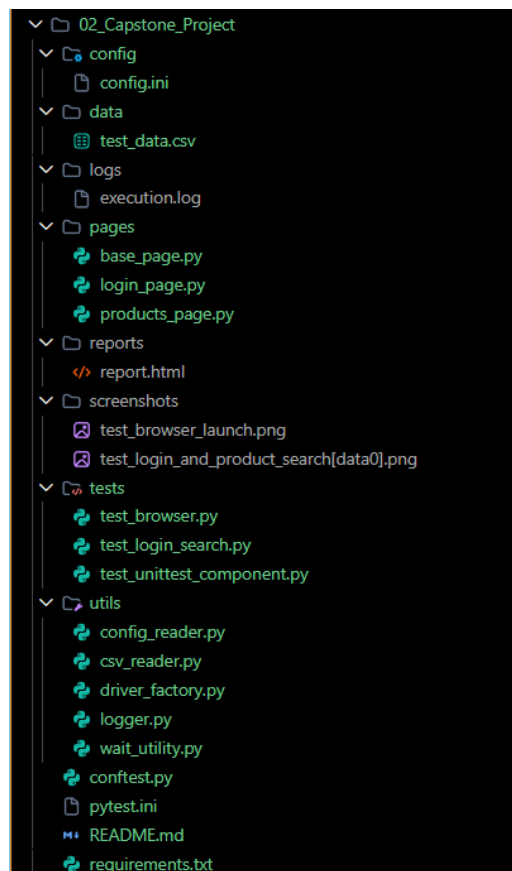


Figure 3. Clean project structure used for the capstone implementation.

6. Configuration Management

Configuration management keeps application and execution settings outside the test logic. The framework reads the base URL, browser name, optional network blocking flag and timeout values from config/config.ini.

```
[application]
base_url = https://automationexercise.com

[browser]
name = chrome
block_ads = true

[timeouts]
explicit_wait = 10
page_load_timeout = 30
```

ConfigReader resolves the configuration file relative to the project structure, reads string values through `get()`, and provides `get_int()` for numeric timeout settings. This allows the test code to remain independent of hard-coded paths.

7. Driver Factory and Browser Setup

DriverFactory centralizes Chrome WebDriver creation. It applies a maximized window, eager page-load strategy, notification/pop-up preferences and the configured page-load timeout. WebDriver Manager is used to obtain the required Chrome Driver.

7.1 Browser Lifecycle

1. The PyTest driver fixture starts a browser session through DriverFactory.
2. The test receives the driver through dependency injection.
3. The test navigates to the configured base URL.
4. After execution, the fixture checks whether the test failed.
5. A screenshot is captured for a failed test before the browser is closed.
6. The driver is then terminated.

7.2 Robustness

The BasePage `safe_click` operation waits for clickability and retries once for transient stale-element or intercepted-click conditions. Explicit waits are used instead of fixed sleep statements.

8. Page Object Model

Page Object Model is used to keep locators and page actions together. Tests therefore express the business flow through methods such as `open_login_page()`, `login()`, `open_products_page()` and `search_product()`, rather than manipulating raw Selenium locators throughout the test.

8.1 LoginPage

- Stores locators for Signup / Login, email, password, login button and logged-in user indicator.
- Provides `login(email, password)` for the login operation.
- Provides `is_logged_in()` for verification.

8.2 ProductsPage

- Stores the Products link and search controls.
- Provides `search_product(product_name)`.
- Checks for the Searched Products heading.
- Checks that the requested product name is present in the returned product elements.

9. Base Page and Wait Utility

Explicit waits improve synchronization between Selenium and the browser. WaitUtility creates a WebDriverWait using the configured timeout and provides reusable conditions for visibility, clickability and presence.

```
def wait_for_visible(self, locator):
    return self.wait.until(
        EC.visibility_of_element_located(locator)
    )

def wait_for_clickable(self, locator):
    return self.wait.until(
        EC.element_to_be_clickable(locator)
    )
```

BasePage builds on the wait utility and provides `safe_click()`, `safe_type()`, `safe_get_text()` and `is_visible()`. This avoids repeating synchronization code across individual Page Objects.

10. Login Automation

The login test opens the configured application URL, navigates to the login page, enters the email from the CSV data, obtains the password from the local environment configuration, clicks Login and verifies the presence of the Logged in as indicator.

10.1 Successful Login Evidence

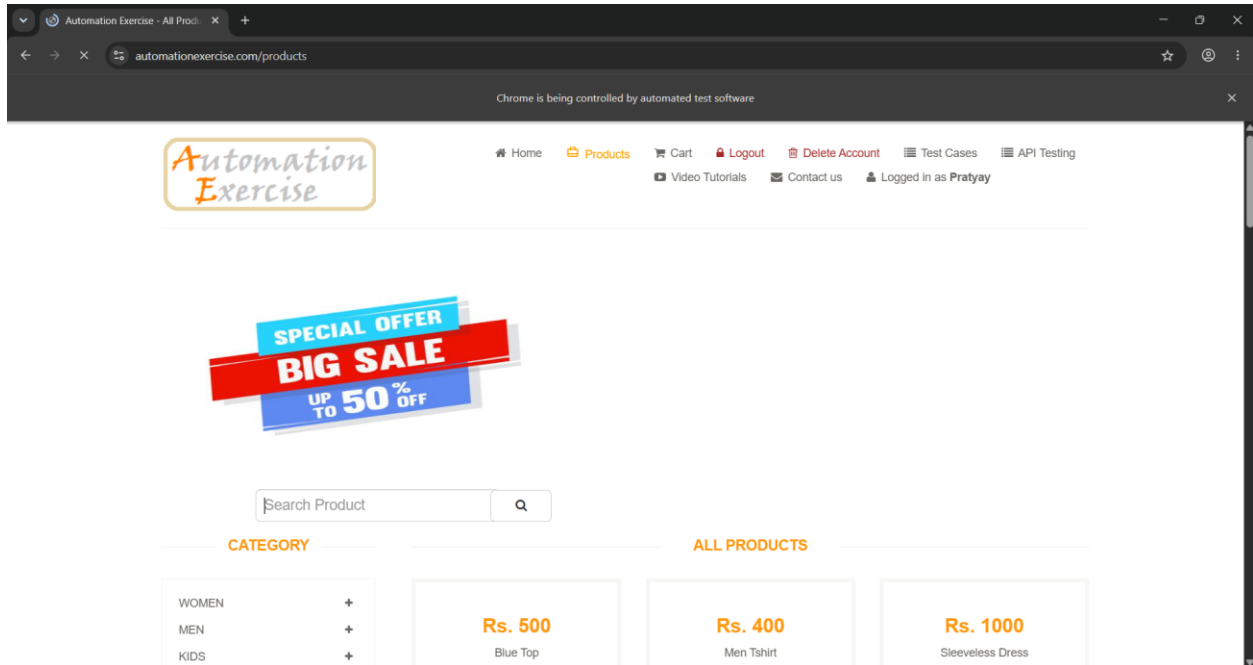


Figure 4. Products page after successful login; the application displays “Logged in as Pratyay”.

Credentials are not stored in the source code. The password is read from the local TEST_PASSWORD environment variable, while the registered email used by the test is stored in the CSV test-data file.

11. Product Search Automation

After successful authentication, the ProductsPage is opened. The search field is populated with the parameterized product name and the search button is clicked. The test verifies both the Searched Products heading and the presence of the requested product in the returned product cards.

11.1 Products Page

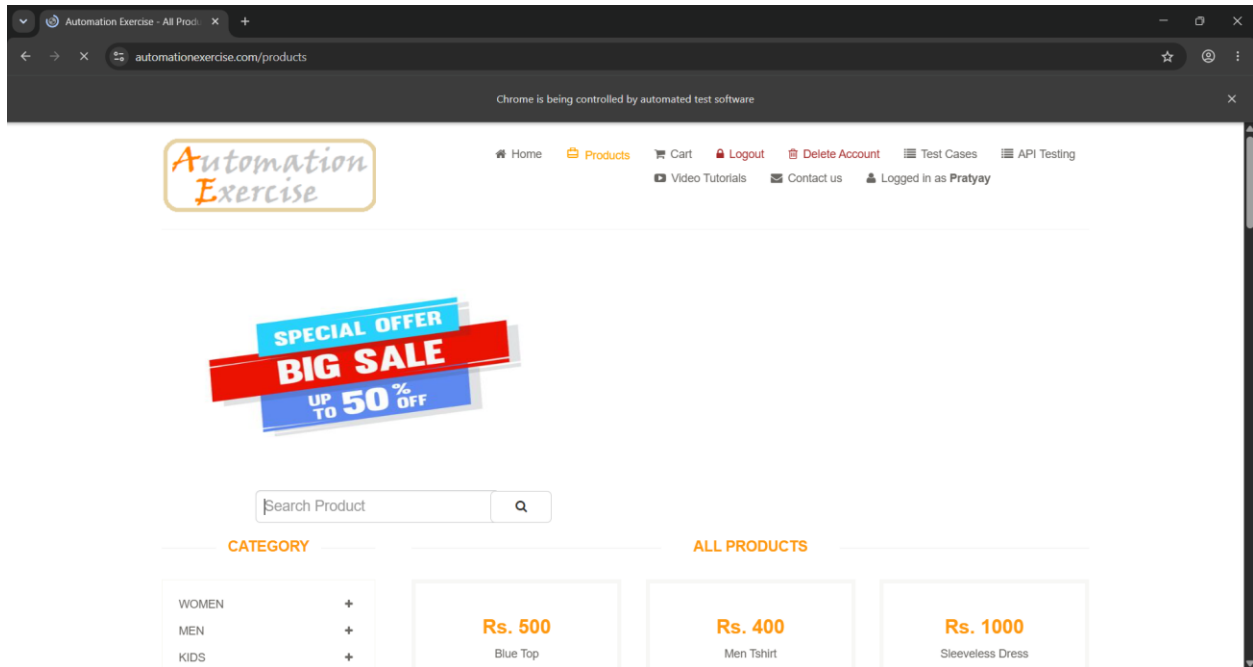


Figure 5. Products page with the search control visible after login.

11.2 Search Result Evidence

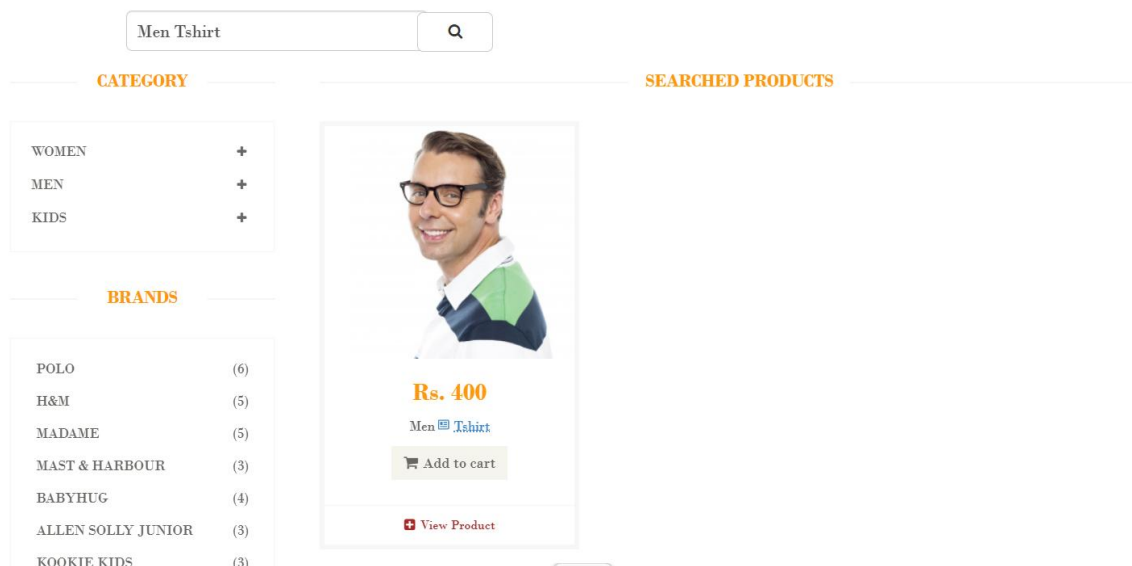
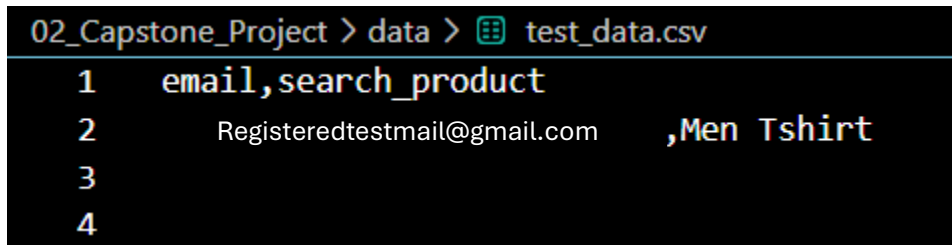


Figure 6. Search for “Men Tshirt” with the Searched Products section and matching result.

12. CSV Test Data Handling

The project uses a CSV file as required by the assignment. CSVReader loads the file through Python's csv module and returns a list of dictionaries. PyTest parameterization then executes the login-and-search test using each row as test input.

```
email,search_product  
REGISTERED_EMAIL,Men Tshirt
```



1	email,search_product
2	Registeredtestmail@gmail.com, Men Tshirt
3	
4	

Figure 7. CSV test data used for parameterized execution (email redacted in the report).

The actual registered email remains local to the project environment. The report intentionally redacts the personal email value while preserving the structure and test-data fields.

13. PyTest and Unittest Integration

PyTest is the primary execution framework. The project uses a fixture in `conftest.py` for browser setup and teardown, parameterization for CSV-driven login/search execution, and `pytest-html` for reporting.

13.1 Unittest Component

The project also contains a Unittest-based `TestConfigReader` class. It validates that the configured base URL matches the intended application and that the explicit wait configuration is greater than zero. This demonstrates the Unittest requirement alongside the PyTest-based browser tests.

```
class TestConfigReader(unittest.TestCase):

    def test_base_url_configuration(self):
        config = ConfigReader()
        self.assertEqual(
            config.get("application", "base_url"),
            "https://automationexercise.com"
        )

    def test_explicit_wait_configuration(self):
        config = ConfigReader()
        self.assertGreater(
            config.get_int("timeouts", "explicit_wait"),
            0
        )
```

14. Logging

The Logger utility creates a project-level execution log and sends messages to both the console and log file. Major events include browser startup, navigation, login, product search, successful verification and test failure screenshot creation.

```
2026-09-25 ... | INFO | Starting browser session
2026-09-25 ... | INFO | Opening AutomationExercise
2026-09-25 ... | INFO | Logging in
2026-09-25 ... | INFO | Login successful
2026-09-25 ... | INFO | Product search successful
```

The logs/ directory is treated as generated output and is excluded from version control through .gitignore.

15. Screenshot on Failure

The PyTest fixture uses the test report object exposed by `pytest_runtest_makereport` to determine whether the call phase failed. When a failure is detected, `save_screenshot()` stores an image in the `screenshots/` directory using the test name.

```
if hasattr(request.node, "rep_call") and request.node.rep_call.failed:
    screenshot_path = screenshot_directory / f"{request.node.name}.png"
    driver.save_screenshot(str(screenshot_path))
```

This mechanism provides visual evidence for debugging without requiring every successful test to capture a screenshot.

16. HTML Reporting

The project configures PyTest to generate a self-contained HTML report using pytest-html. The report contains the test summary, environment information, individual test results and execution durations.

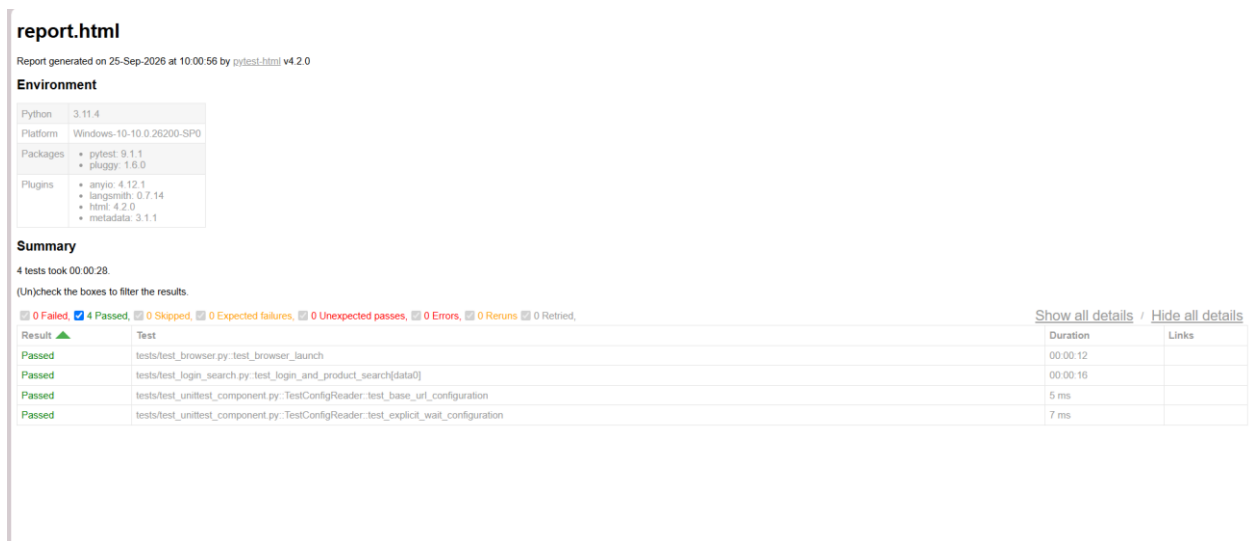


Figure 8. Generated PyTest HTML report showing four passed tests.

The report is generated at `reports/report.html` according to `pytest.ini`. The latest captured report shows 4 passed tests, 0 failed, 0 skipped and 0 errors.

17. Test Cases

ID	Test Case	Method	Expected Result	Status
TC-01	Browser launch and application title	PyTest	Automation Exercise loads and title is correct	PASS
TC-02	Login and product search	PyTest + parameterization	Login succeeds and Men Tshirt appears in search results	PASS
TC-03	Base URL configuration	Unittest	Configured URL equals application URL	PASS
TC-04	Explicit wait configuration	Unittest	Configured wait timeout is greater than zero	PASS

17.1 Test Flow

7. Create browser session using DriverFactory.
8. Navigate to Automation Exercise.
9. Open Login page.
10. Enter CSV email and environment-provided password.
11. Verify Logged in as indicator.
12. Open Products page.
13. Search for Men Tshirt.
14. Verify Searched Products heading.
15. Verify Men Tshirt is present in returned products.
16. Close browser and record the result.

18. Execution Results

The latest complete execution was performed from the 02_Capstone_Project directory using the pytest command. All four collected tests passed.

```
PS C:\Users\pratyay chowdhury\OneDrive\Desktop\Selenium-python-automation\02_Capstone_Project> pytest
===== test session starts =====
platform win32 -- Python 3.11.4, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\pratyay chowdhury\AppData\Local\Programs\Python\Python311\python.exe
cachedir: .pytest_cache
metadata: {'Python': '3.11.4', 'Platform': 'Windows-10-10.0.26200-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy': '1.6.0'}, 'Plugins': {'anyio': '4.12.1', 'langsmith': '0.7.14', 'html': '4.2.0', 'metadata': '3.1.1'}}
html: '4.2.0', 'metadata': '3.1.1'}}
rootdir: C:\Users\pratyay chowdhury\OneDrive\Desktop\Selenium-python-automation\02_Capstone_Project
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.12.1, langsmith-0.7.14, html-4.2.0, metadata-3.1.1

tests/test_browser.py::test_browser_launch PASSED [ 25%]
tests/test_login_search.py::test_login_and_product_search[data0] PASSED [ 50%]
tests/test_unittest_component.py::TestConfigReader::test_base_url_configuration PASSED [ 75%]
tests/test_unittest_component.py::TestConfigReader::test_explicit_wait_configuration PASSED [100%]

- Generated html report: file:///C:/Users/pratyay%20chowdhury/OneDrive/Desktop/Selenium-python-automation/02_Capstone_Project/reports/report.html -
4 passed in 37.69s
```

Figure 9. Terminal execution showing 4 passed in 37.69 seconds and HTML report generation.

18.1 Result Summary

Metric	Result	Observation
Collected tests	4	Four tests executed in the final suite
Passed	4	All collected tests passed
Failed	0	No failures in final execution
Skipped	0	No tests skipped
Execution time	37.69 s	Latest terminal execution
HTML report	Generated	reports/report.html

19. Challenges and Solutions

Challenge	Framework Response
Dynamic browser timing	Used explicit waits through WaitUtility instead of fixed delays.
Repeated Selenium interactions	Moved common click/type/visibility behavior into BasePage.
Maintainability of locators	Used Page Object Model with dedicated LoginPage and ProductsPage classes.
Configuration duplication	Centralized URL and timeout values in config.ini.
Test data duplication	Loaded login/search inputs from CSV and used PyTest parameterization.
Failure diagnosis	Added automatic failure screenshots and structured logging.
Reporting	Configured pytest-html to generate a self-contained HTML report.

20. Security and Good Practices

- Passwords are not hard-coded into the test source; TEST_PASSWORD is read from the local environment.
- Generated reports, screenshots and logs are excluded from Git through .gitignore.
- Python cache directories are excluded from version control.
- The framework avoids unnecessary fixed sleep statements and uses explicit waits.
- Page interactions are centralized to reduce duplicated locators and actions.
- The report redacts the personal email value used in the CSV screenshot.

20.1 Repository Organization

The official student instruction notice requires an individual GitHub repository and specifies that the Capstone Project folder should contain complete source code, project report, outputs, screenshots, README and demonstration video or link.

[filecite2turn3file1](#)

The current project is organized under the capstone folder with configuration, data, pages, tests, utilities and evidence directories, plus the README, requirements and pytest configuration files.

21. Conclusion

The Selenium Python Automation Framework successfully implements the assigned Login and Product Search scenario using a reusable project structure. The framework demonstrates PyTest execution, Unittest validation, Page Object Model, reusable utilities, configuration management, CSV test data, failure screenshot handling, logging and HTML reporting—the major framework elements specified in the capstone brief. [filecite turn3file0](#)

The final recorded execution produced four passing tests with no failures. The evidence captured in this report demonstrates the application flow, project structure, test data, successful search result, terminal execution and generated HTML report.

21.1 Result

A screenshot of a terminal window with a black background and green text. The text reads: `= 4 passed in 28.38s ==`

FINAL TEST RESULT: 4 / 4 TESTS PASSED

22. Future Scope

The current implementation intentionally stays within the assigned Login and Product Search scope. If the framework is extended in a later project, possible enhancements include:

- Cross-browser execution for additional browser engines.
- Parallel test execution for larger test suites.
- CI/CD execution through a build pipeline.
- Additional page objects and broader functional coverage.
- Centralized test environment selection for multiple environments.
- Richer reporting with historical execution trends.
- Externalized test-data management for larger datasets.

23. References

17. Course Capstone Assignment – Selenium Python Framework Development (Unittest + PyTest + POM), provided as part of the Selenium Python Automation course.
18. Official Student Instruction Notice – Selenium Python Automation, including GitHub and Capstone Project submission requirements.
19. Automation Exercise – public e-commerce practice application used as the Application Under Test.
20. Selenium WebDriver – browser automation library used by the framework.
21. PyTest and pytest-html – test execution and HTML reporting tools used by the project.

Note: The project report is based primarily on the supplied course assignment and instruction documents and on the actual implementation and execution evidence captured from the project.

Appendix A – Key Framework Configuration

```
[pytest]
addopts = -v --html=reports/report.html --self-contained-html
testpaths = tests
```

Appendix B – Submission Checklist

- Source code present in 02_Capstone_Project.
- README.md included.
- requirements.txt included.
- Project report included.
- Screenshots/evidence prepared.
- HTML execution report generated.
- Generated cache folders excluded from version control.
- Sensitive credentials kept outside source code.
- GitHub repository organized according to the course notice.